POLITECNICO
MILANO 1863

Software Engineering 2

Introduction to Requirements Engineering (RE)

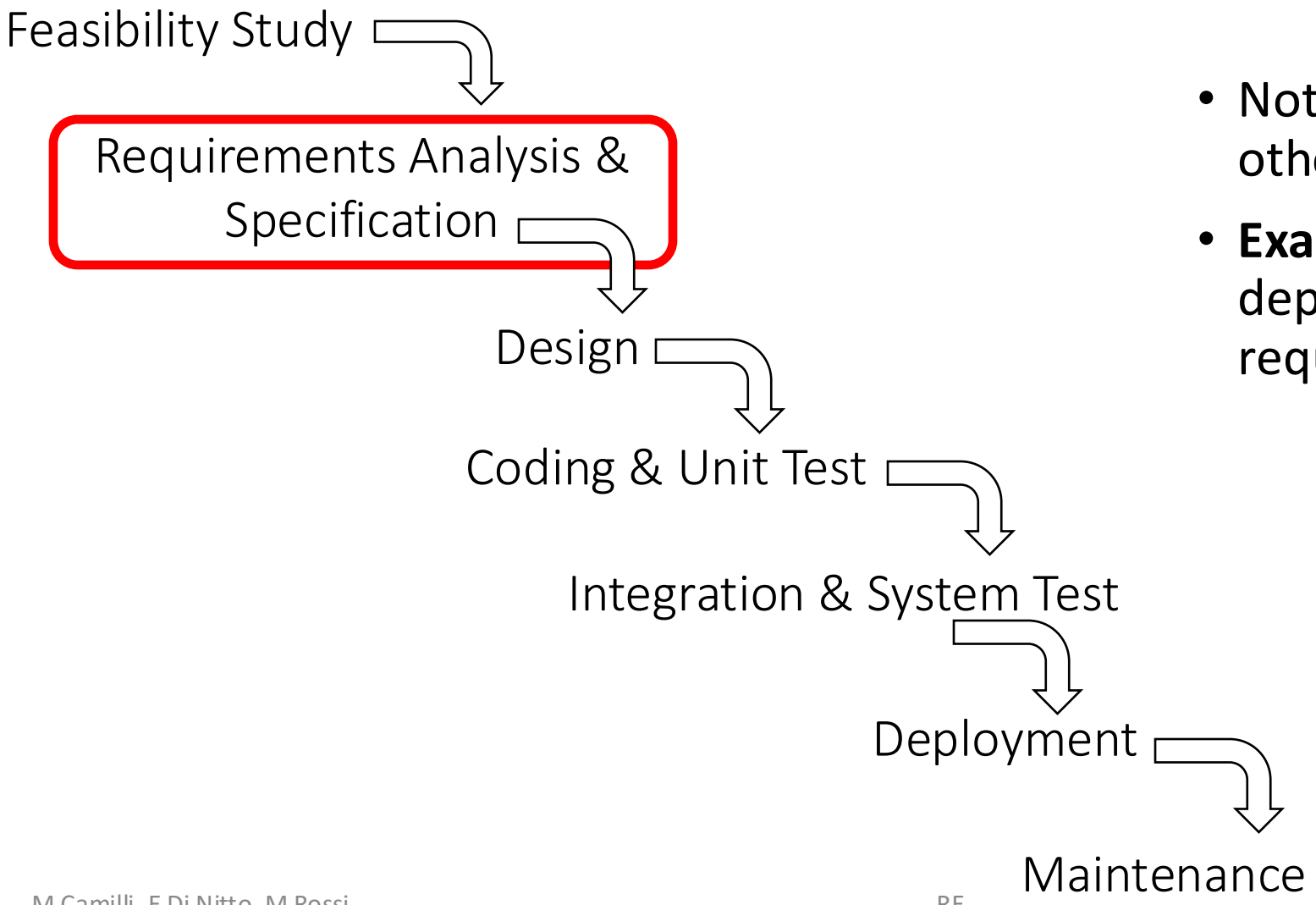
The world and the machine

The Airbus incident

Requirements Engineering (RE)

Context, Definitions, Importance and difficulties, RE process

Context: where do we find RE?



- Not to be forgotten in all other phases, too!
- **Example**: as the system is deployed, new requirements emerge!

Requirements Engineering: definition

[Nuseibeh&Easterbrook '00]

- The primary measure of success of a software system is the degree to which it meets the purpose for which it was intended
- Software systems RE is the process of discovering that purpose, by identifying stakeholders and their needs, and documenting these in a form that is amenable to analysis, communication, and subsequent implementation
- Important issues
  - Identify stakeholders
  - Identify their needs
  - Produce documentation
  - Analyse, communicate, implement requirements

What is a requirement?

POLITECNICO MILANO 1863

- Examples of candidate requirements
  - “The system shall allow users to reserve taxis”
  - “The system has to provide a feedback in 5 seconds”
  - “The system should never allow non-registered users to see the list of other users willing to share a taxi”
  - “The system should be available 24/7”
  - “The system should guarantee that the reserved taxi picks the user up”
  - “The system should be implemented in Java”
  - “The search for the available taxi should be implemented in class Controller”

Types of requirements

POLITECNICO MILANO 1863

- Functional Reqs: describe the **interactions** between the **system** and its **environment** (independent from implementation)
  - Examples:
    - "The word processor shall allow users to search for strings in the text"
    - "The system shall allow users to reserve taxis"
  - Are the main (functional) goals the software has to realize
- Non-functional Reqs (NFRs): further characterization of user-visible aspects of the system not directly related to functions
  - Examples:
    - "The response time must be less than 1 second"
    - "The server must be available 24 hours a day"
- Constraints (or technical requirements): imposed by the customer or the environment in which the system operates
    - "The implementation language must be Java"
    - "The credit card payment system must be able to be dynamically invoked by other systems relying on it"

NFRs and product qualities

- NFRs predicate over external non-functional qualities
- Must be measurable through metrics

Software Product Quality

Functional suitability
- Functional completeness
- Functional correctness
- Functional appropriateness

Performance efficiency
- Time behavior
- Resource utilization
- Capacity

Reliability
- Faultlessness
- Availability
- Fault tolerance
- Recoverability

Security
- Confidentiality
- Integrity
- Non-repudiation
- Accountability
- Authenticity
- Resistance

Compatibility
- Co-existence
- Interoperability

Safety
- Ability to set operational constraints
- Ability to identify risks
- Ability to fail safe
- Ability to warn about hazards
- Ability to safe integration

Mantainability
- Modularity
- Reusability
- Analyzability
- Modifiability
- Testability

Flexibility
- Adaptability
- Scalability
- Installability
- Replaceability

Interaction capability
- Appropriateness recognizability
- Learnability
- Operability
- User error protection
- User engagement
- Inclusivity
- User assistance
- Self-descriptiveness

Characteristics of NFRs

POLITECNICO MILANO 1863

- Constraints on how functionality must be provided to the end user
- Application domain determines
  - Their relevance
  - Their prioritization
- **Examples**:
  - Relevant nonfunctional requirements for Netflix?
  - Relevant nonfunctional requirements for Ariane 5?
- Have a strong impact on the structure of the system to be
  - Example: a system requires 24/7 availability → it is likely to be thought as a replicated system (with redundant components)

Examples of bad requirements

- "The system shall validate and accept credit cards and cashier's check with high priority"
- **Issue(s)?**
  - Multiple concerns: two requirements instead of one
  - Ambiguous: if the credit card processing works, but the cashier's check validation does not, is this requirement satisfied?
    - Should not, but this is misleading…
  - Ambiguous: does "high priority" refer to cashier's checks only, so are credit cards low priority? Other way around? Are they both high priority?

Examples of bad requirements

POLITECNICO MILANO 1863

- "The system shall process all mouse clicks very fast to ensure users do not have to wait"
- **Issue(s)?**
  - Cannot be verified (tested): what does "fast" mean? Do we have a metric? Can you quantify it?

Examples of bad requirements

POLITECNICO MILANO 1863

- “the user must have Adobe Acrobat installed”
- **Issue(s)?**
  - Cannot be achieved by the software system itself: it is not something the system must do
  - It could be expressed as a constraint or assumption, but it is not a functional requirement

Exercise

POLITECNICO
MILANO 1863

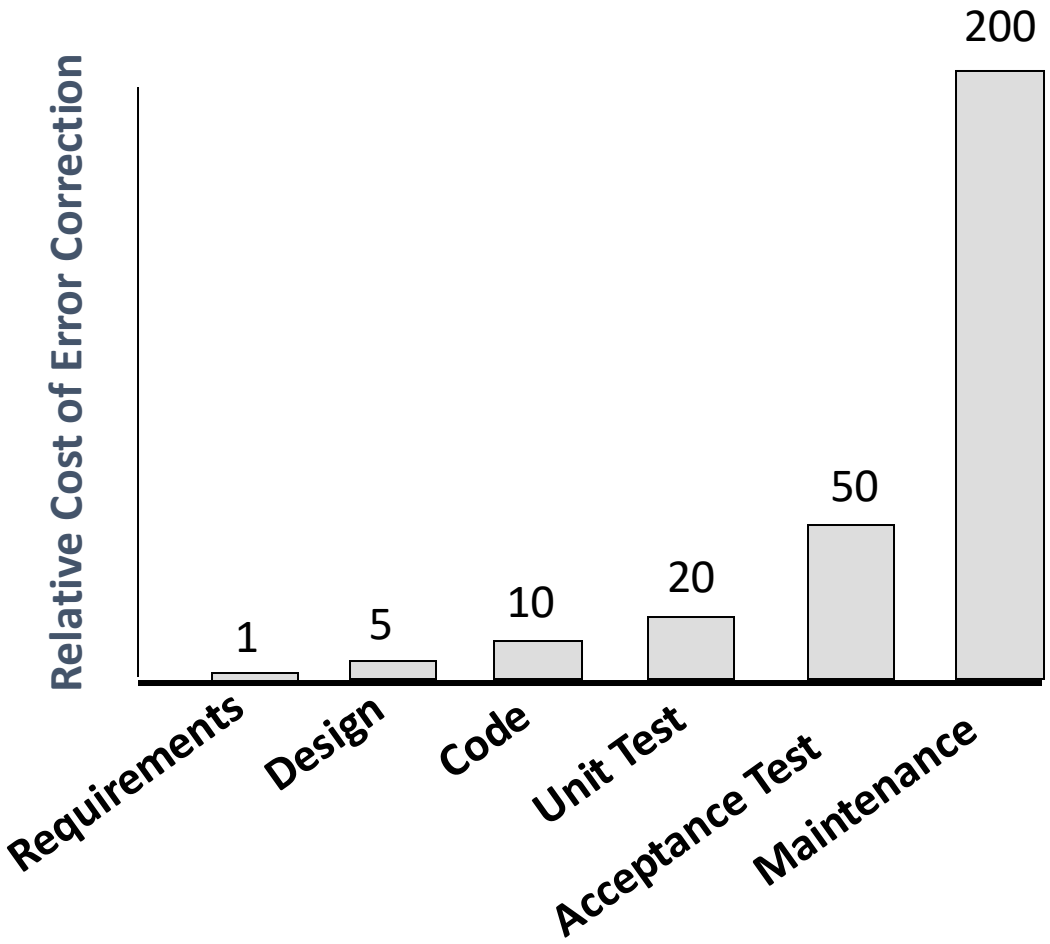
- Consider the following requirements:
  - "The system shall allow users to reserve taxis"
  - "The system has to provide a feedback in 5 seconds"
  - "The system should never allow non-registered users to see the list of other users willing to share a taxi"
  - "The system should be available 24/7"
  - "The system should guarantee that the reserved taxi picks the user up"
  - "The system should be implemented in Java"
  - "The search for the available taxi should be implemented in class Controller"
- Classify them by their kind (functional, nonfunctional and constraints)
- Highlight bad requirements

Issues concerning RE

- Poor requirements are ubiquitous ...
  - “The system should guarantee that the reserved taxi picks the user up”: is this reasonable?
  - “[…] requirements need to be engineered and have continuing review & revision” (Bell & Thayer, empirical study, 1976)
- RE is hard & critical ….
  - “[…] hardest, most important function of SE is the iterative extraction & refinement of requirements” (F. Brooks, 1987)

Cost of late correction (Boehm, 1981)

- The cost of correcting an error depends on the number of subsequent decisions that are based on it
- Errors in requirements have the potential for greatest cost, because many other design decisions depend on them



What makes RE so complex ?

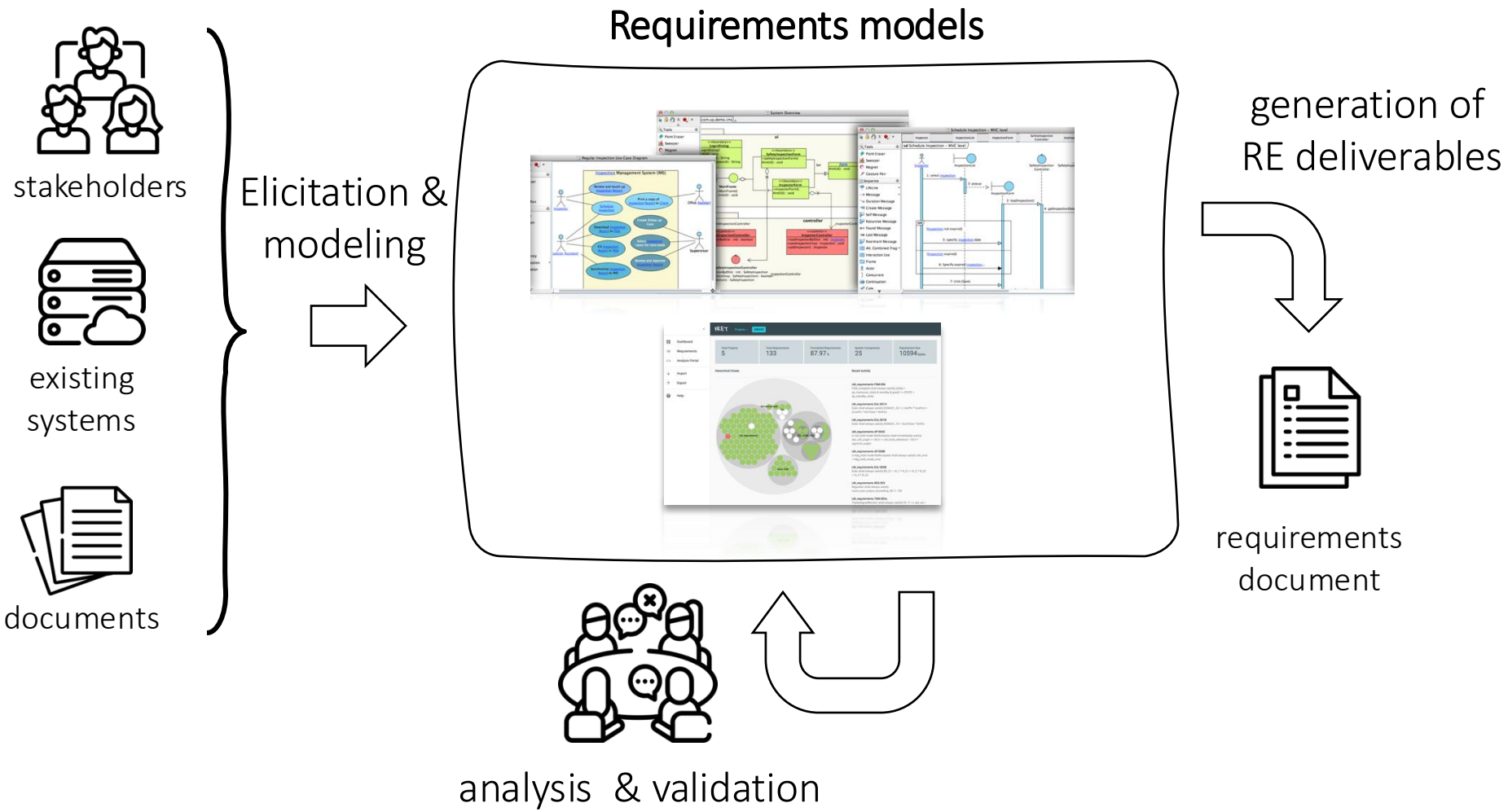
POLITECNICO MILANO 1863

- Broad scope
  - **Composite systems**
    - human organizations + physical devices + software components
  - **More than one system**
    - alternative proposals for system-to-be, system evolutions, product family
  - **Multiple abstraction levels**
    - high-level goals, operational details
- Multiple concerns
  - functional, quality, development
  - hard and soft concerns

  $\Longrightarrow$ conflicts
- Multiple stakeholders with different background
  - customers, users, domain experts, developers, ...

  $\Longrightarrow$ conflicts

RE workflow


stakeholders
existing systems
documents
Elicitation & modeling
Requirements models
generation of RE deliverables
requirements document
analysis & validation

Requirements Engineering (RE)

Understanding phenomena and requirements: the World and the Machine

Example: ambulance dispatching system

- For every urgent call reporting an accident, an ambulance should arrive at the incident location within 14 mins
- For every urgent call, details about the accident are correctly encoded
- When an ambulance is dispatched, it will reach the accident location in the shortest possible time
- Accurate ambulance locations are known by GPS
- Ambulance crews correctly notify ambulance availability through a mobile data terminal

Examples of open questions

POLITECNICO
MILANO 1863

- Are you able to elicit requirements out of this description?
- **Possible questions**
  - Should the software system drive the ambulance?
  - What are the "correctly encoded" details about incidents?
  - Do terminals exist already or not?
  - …
- More in general
  - What is the boundary of the system? What is inside/outside? What is in-between?
  - How do we reason about these aspects in a systematic way?

The World and the Machine
(M. Jackson & P. Zave, 1995)

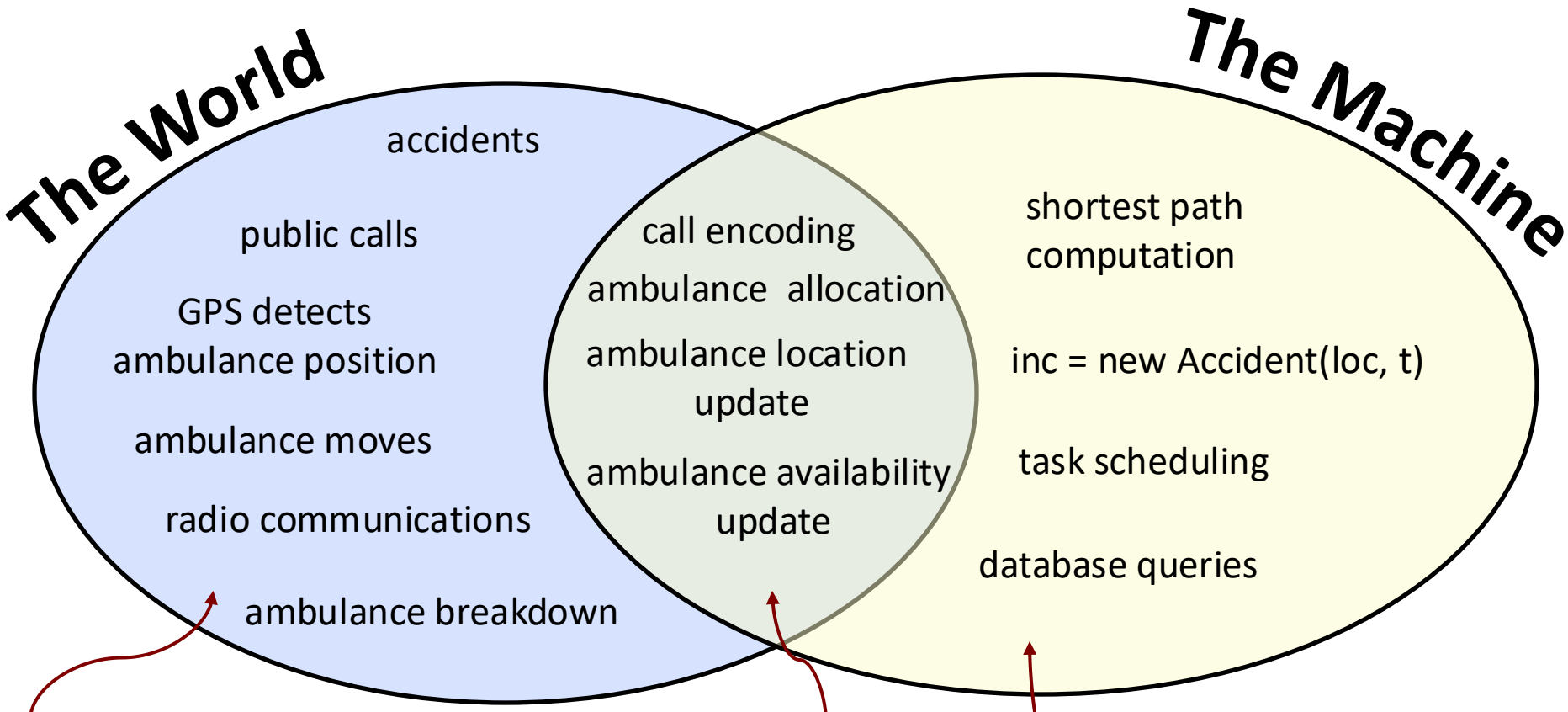
POLITECNICO MILANO 1863

- Terminology
  - **Machine** = the portion of system to be developed
    - typically, software-to-be + hardware
  - **World** (or environment) = the portion of the real-world affected by the machine
- The purpose of the machine is always in the world
  - Examples
    - Ambulance Dispatching System
    - Banking Application
    - Word Processor
    - …

World and machine phenomena

- RE is concerned with phenomena occurring in the world
  - For an ambulance dispatching system:
    - Occurrences of accidents
    - Report of accidents by public calls
    - Encodings of calls' details into the dispatching software
    - Allocation of an ambulance
    - Arrival of an ambulance at the accident location
- As opposed to phenomena occurring inside the machine
  - For the same ambulance dispatching system:
    - the creation of a new object of class `Accident`
    - the update of a database entry
- **Requirement models are models of the world!**

The ambulance dispatching system

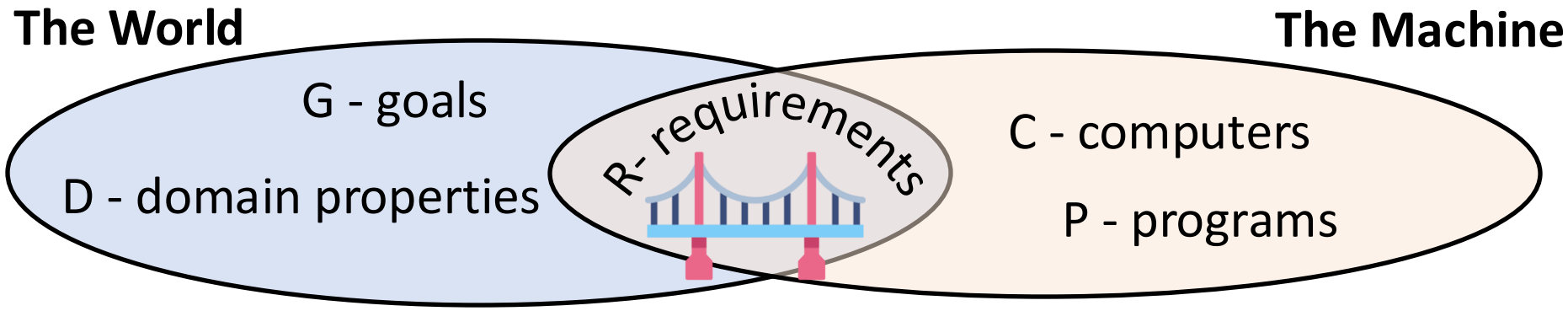


world phenomena the machine cannot observe

shared phenomena

phenomena located entirely in the machine

Goals, domain assumptions, requirements



- Goals → **prescriptive** assertions formulated in terms of world phenomena (not necessarily shared)
- Domain properties/assumptions → **descriptive** assertions **assumed to hold** in the world
- Requirements → **prescriptive** assertions formulated in terms of **shared** phenomena

Goals, domain assumptions, requirements: an example

- **Goal**
  - The emergency response authority would like to dispatch an ambulance for every accident report received, such that it arrives at the accident scene within 15 minutes
- **Domain assumptions**
  - For every call reporting an accident, details about the accident are correctly encoded
  - When an ambulance is dispatched, it will reach the accident location in the shortest possible time
  - Location updates are accurately provided by ambulance GPS systems
  - Ambulance crews correctly provide notification of ambulance availability through an existing mobile data terminal
- **Requirement**
  - When a call reporting a new accident is encoded, the system shall dispatch the nearest available ambulance according to information available from location updates and availability notifications

Completeness of Requirements

POLITECNICO MILANO 1863

- Given the set of requirements R, goals G and domain assumptions D
- **R** is complete iff
  - **R** ensures satisfaction of **G** in the context of domain assumptions **D**

$$\mathbf{R \text{ and } D \models G}$$

    - Analogy with program correctness: a Program P running on a particular Computer C is correct if it satisfies the Requirements R
      - P and C $\models$ R
  - **G** captures all the stakeholders' needs
  - **D** represents valid properties/assumptions about the world

Requirements Engineering (RE)

What can go wrong when defining G, R, D?

The A320 example

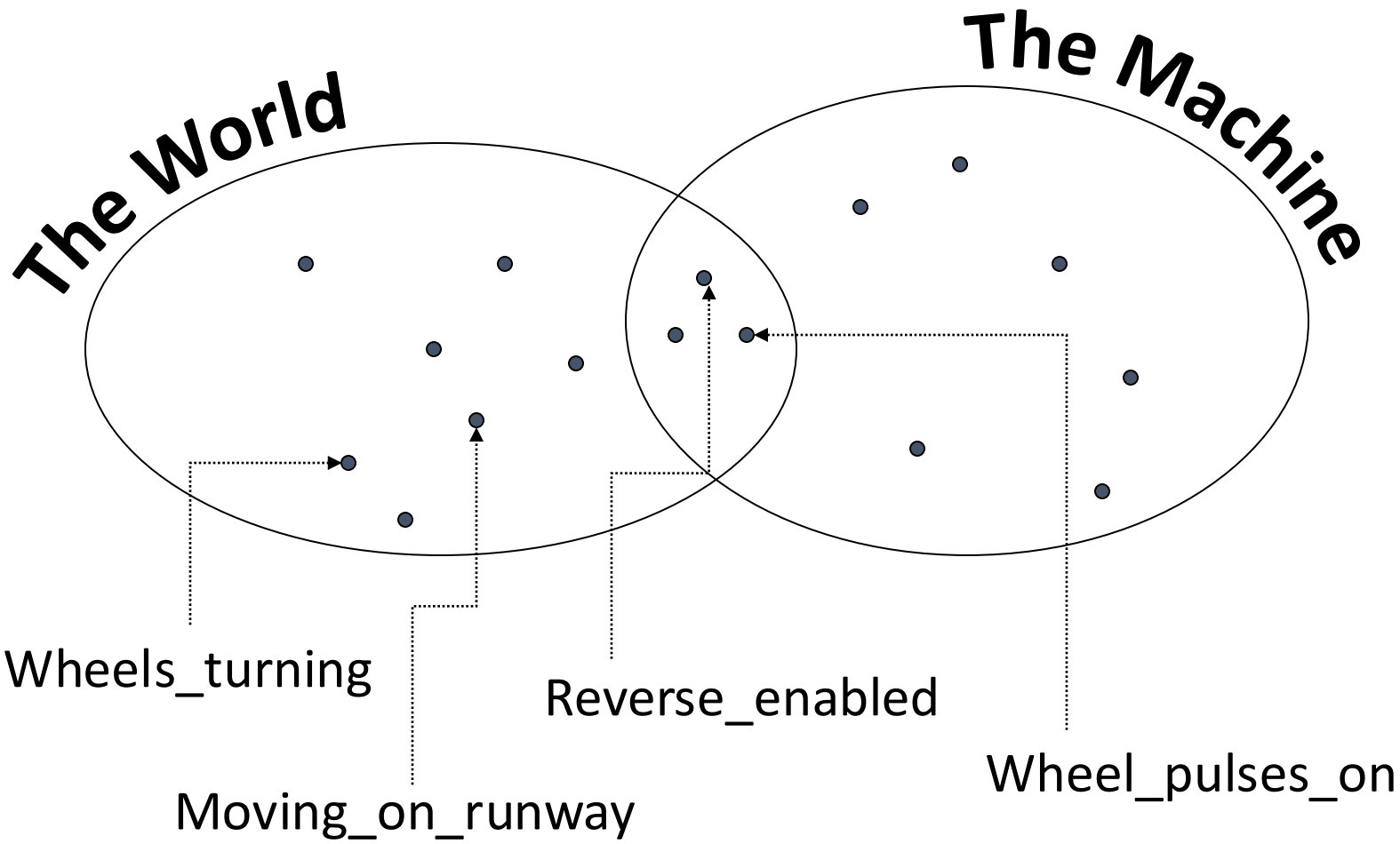
Example – Airbus A320

https://aviation-safety.net/database/record.php?id=19930914-2

- [Sep 14, 1993] A Lufthansa Airbus on a flight from Frankfurt to Warsaw landed in bad weather conditions (rain and wind)
- On landing, the aircraft's software-controlled braking system deployed with a delay of 9 seconds after touch down
- There was insufficient runway remaining to stop the plane and the aircraft ran into a grass embankment
- **Result**: 2 people were killed, 54 injured
- **Several causes**:
  - Human errors
  - Software errors (braking control system)

Lufthansa Airbus A.320-211 D-AIPN at Frankfurt 27 May 1990 , © Werner Fischdick

Example – Airbus A320 Braking Logic



Goal, domain assumptions, requirement

- **Goal G**
  - “Reverse thrust enabled iff the aircraft is moving on the runway”
- **Domain Assumptions D**
  - “Wheel pulses on iff wheels turning”
  - “Wheels turning iff moving on runway”
- **Requirement R**
  - “Reverse thrust enabled iff wheel pulses on”

- **Verification**: R and D $\models$ G holds

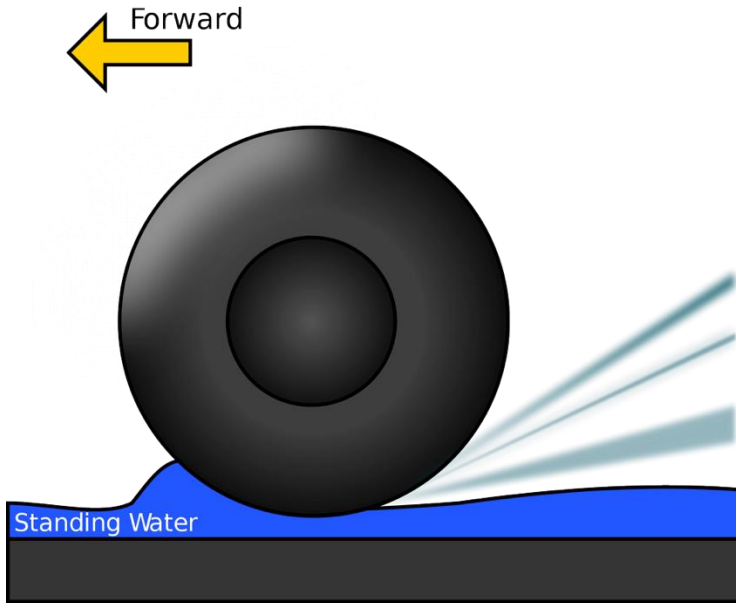
Correctness argument

- Goal
  - Reverse_enabled ⇔ Moving_on_runway
- Domain properties
  - Wheel_pulses_on ⇔ Wheels_turning
  - Wheels_turning ⇔ Moving_on_runway
- Requirement
  - Reverse_enabled ⇔ Wheels_pulses_on

- We can prove that $R \wedge D \models G$ → it seems the system is correct by design
  - What happened?
  - D are not valid assumptions!
  - **Invalid domain assumptions → Warsaw accident**

Problem: Aquaplaning

- Domain assumptions D do not hold in reality, the correctness argument is flawed!
- Incorrect domain assumptions lead to disasters!



Requirements Engineering (RE)

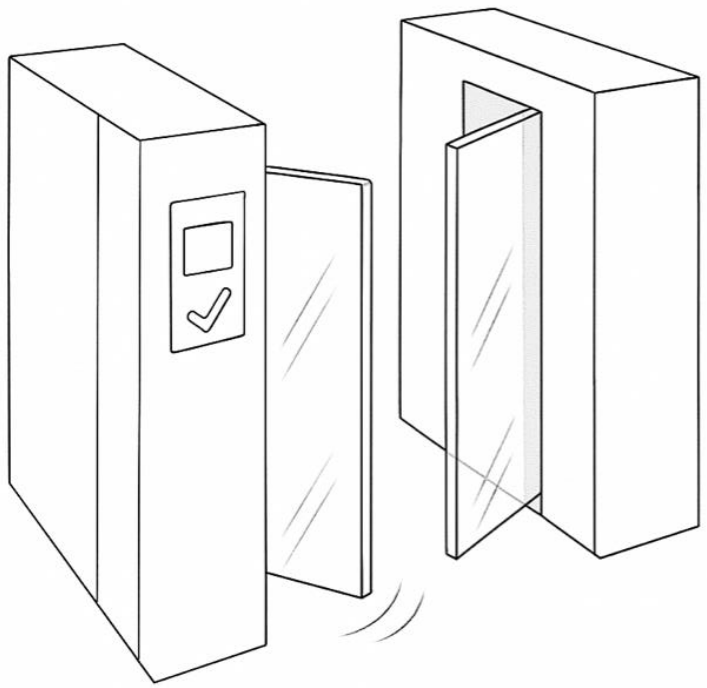
Example of goals, assumptions, requirements

Physical Access Control System (PACS)

Inspired by M. Jackson and P. Zave, "Deriving Specifications from Requirements: an Example", 1995

- System for regulating the flow of people accessing a "restricted" area
  - e.g., a barrier at the entrance of a museum
- Doors swing open to allow people in, and prevent people from entering if they do not have the right to do so
- People gain access to the restricted area by buying "tokens"
  - each token is good for a single entrance
- Tokens can be "materialized" in different ways (paper tickets, special-purpose coins, electronic tickets)
- To enter the area, people push the doors of the barrier after scanning a token (the doors swing open if the token is valid).

- The ultimate purpose of the barrier is to allow paying customers to enter the area, and to keep out people who have not paid.



Next steps

1. Carry out domain analysis
   - Identify phenomena, in particular the shared ones
   - Identify goals
   - Identify domain assumptions
2. Define requirements
   - … with an eye towards the goals to be achieved
   - Make sure that requirements are expressed using only shared phenomena
3. Prove that R and D $\models$ G
   - i.e., requirements + domain assumptions guarantee goal satisfaction

Phenomena

- The visitor buys a token
- The visitor scans a token
- The token is recognized as valid
- The visitor pushes the barrier doors
- The doors swing open
- The visitor enters the exhibition area
- The doors close

Phenomena

- The visitor buys a token
- The visitor scans a token
- The token is recognized as valid
- The visitor pushes the barrier doors
- The doors swing open
- The visitor enters the exhibition area
- The doors close

world phenomena
shared phenomena
  environment controlled
  machine controlled
machine phenomena

Goals

G1: At any time, entries should never exceed the number of scanned tokens

G2: People who scan tokens are allowed to enter

- **Note**
  - They are **prescriptive** descriptions
  - G1 is said to be safety property—the form is “nothing bad will ever occur”

Domain assumptions

DA1: Visitors only enter the exhibition area when the doors are open

DA2: If a visitor pushes the barrier and the doors open, they enter the exhibition area

DA3: When the doors are open, it is not possible for two visitors to enter the exhibition area with a delay of less than d_e time units between them, where d_e is the time needed by the system to close the doors

- For simplicity:

DA4: The time that the mechanical system takes to open and close the doors is negligible

Let us revisit goal G1

G1: At any time, entries (*e*) should never exceed the number of scanned tokens (*s*)

- i.e., ***e* ≤ *s* must hold**

- **G1** can be enforced by controlling either entries or the scanning of tokens
  - the machine cannot force scanning events
  - it can prevent entering events (by not opening the doors)
    - Entering and door openings are linked through DA1 and DA2 and "pushing" events

Requirements for G1

G1: ***e* ≤ *s* must hold**

R1: When a visitor pushes the barrier doors, if the number of times that the door has opened (*o*) is less than the number of tokens scanned (*s*), then the system shall open the doors

- i.e., only if ***o* < *s* holds**

R2: When a visitor pushes the barrier doors, if the number of times that the door has opened is equal to the number of tokens scanned, then the system shall keep the doors closed

- i.e., if ***o* = *s* holds**

R3: The system shall open the doors only if a visitor pushes them

R4: After the doors open, the system shall close the doors within d_e time units

- i.e., before the next person can enter

R5: Initially, the system shall close the doors

Show that G1 holds (1)

- We show that R1, R2, R3, R4, R5, DA1, DA2, DA3, DA4 guarantee G1
  - R1, R2, R3, R4, R5, DA1, DA2, DA3, DA4 $\models$ G1
- **First**, we show that the number of entries is not greater than the number of times the doors open, i.e., $e \leq o$ holds
  - From R5 and DA1, initially no one can enter the exhibition area, unless the doors open
  - From DA2 and R3, each new opening of the doors corresponds to a visitor entering the exhibition area, where the opening occurs before the entering
  - From R4, DA3 and DA4, only one visitor enters when the doors open
  - Thus, $e \leq o$ holds at any time

Show that G1 holds (2)

- **Second**, we show that the number of times that the doors open is not greater than the number of tokens scanned, i.e., $o \leq s$ holds
  - Tokens can be scanned only after the system starts, so initially $s = 0$
  - From R3, the doors do not open if there is no push
  - R1, R2 and R3 together entail that the doors open if, and only if, a visitor pushes the doors and the number of scanned tokens is greater than the number of times that the doors have opened
    - i.e., only if $o < s$ holds
  - Hence, $o \leq s$ holds at all times
- Finally, $e \leq o \leq s$ holds $\Longrightarrow$ **G1 is satisfied**

Exercise

- Reason on G2 and determine whether R1-R5 and DA1-DA4 (or a subset of these) are sufficient to entail G2

Assignments for the next week classes

- Watch the videos you find here
  - On requirement elicitation: https://polimi365-my.sharepoint.com/:v:/g/personal/10143828_polimi_it/EQzGESnSzdBMnTIQvI9XG6QBV0zp715j6HqGqhHJ2Vx7qQ?e=Rx4Cab
  - On modeling: https://polimi365-my.sharepoint.com/:v:/g/personal/10143828_polimi_it/EfuNIeNgPwtIrwcJQNI1s2YBw6G0vEhQDw4yNbpqLH4wQg?e=per3xm
- They will be used as a basis for discussion in class next time(s)

Summary

POLITECNICO MILANO 1863

- The boundary between the World & the Machine is generally not given at the beginning of the development process
- The purpose of a RE activity is
  - to identify the real goals of the project
  - to explore alternative ways to satisfy the goals, through:
    - alternative interfaces between the world and the machine
    - alternative pairs (R, D) such that
      R and D $\vDash$ G
  - to evaluate the strengths and risks of each alternative, in order to select the most appropriate one

References

- B. Paech, "What Is a Requirements Engineer?" in IEEE Software, vol. 25, no. 04, pp. 16-17, 2008.
- "ISO/IEC/IEEE International Standard - Systems and software engineering -- Life cycle processes -- Requirements engineering," in ISO/IEC/IEEE 29148:2018(E) , vol., no., pp.1-104, 30 Nov. 2018, doi: 10.1109/IEEESTD.2018.8559686.
- Bashar Nuseibeh and Steve Easterbrook. 2000. "Requirements engineering: a roadmap". In Proceedings of the Conference on The Future of Software Engineering (ICSE '00). Association for Computing Machinery, New York, NY, USA, 35–46. DOI:https://doi.org/10.1145/336512.336523
- P. Zave, "Classification of research efforts in requirements engineering," Proceedings of 1995 IEEE International Symposium on Requirements Engineering (RE'95), 1995, pp. 214-216, doi: 10.1109/ISRE.1995.512563.
- Michael Jackson. 1995. "The world and the machine". In Proceedings of the 17th international conference on Software engineering (ICSE '95). Association for Computing Machinery, New York, NY, USA, 283–292. DOI:https://doi.org/10.1145/225014.225041
- M. Jackson and P. Zave, "Deriving Specifications from Requirements: an Example," 1995 17th International Conference on Software Engineering, 1995, pp. 15-15, doi: 10.1145/225014.225016.